

Gradious AI-Enhanced Cloud File Storage Platform

Project Technical Documentation & Architecture Report

1. Problem Statement

Traditional cloud file storage platforms often store user assets as opaque binary blobs without understanding their underlying contents or tracking user security risks. Users struggle to organize, summarize, and extract key insights from large documents, code repositories, or media files without manual analysis. Furthermore, existing cloud platforms frequently suffer from insecure file-sharing mechanics (e.g., exposing sequential database IDs) and lack automated, real-time detection of suspicious upload behaviors or malicious payloads.

2. Approach & Solution Strategy

To solve these challenges, Gradious introduces an AI-native cloud storage and sharing ecosystem combining three core pillars:

- **Full-Stack Storage Core:** FastAPI backend with MySQL database providing secure JWT authentication, folder hierarchy, unpredictable UUID share links, storage quota tracking, and comprehensive activity logging.
- **Machine Learning Module (Random Forest):** Scikit-learn Random Forest classifiers for automated multi-class file classification (Document, Image, Video, Audio, Code) and anomaly detection for identifying suspicious upload behaviors.
- **Generative AI Module (Google Gemini 3.6 Flash):** Automated document text extraction (PDF, DOCX, TXT, CSV, JSON, MD, Code) and direct multimodal analysis for media assets to produce smart summaries, descriptions, tags, and content insights.

3. System Architecture

Layer	Technologies	Key Responsibilities
Frontend	HTML5, Vanilla CSS3, JavaScript ES6+	Responsive UI, upload forms, AI details display, storage dashboard, link sharing.
Backend	Python 3.11, FastAPI, Uvicorn	REST APIs, JWT authorization, file I/O sanitization, path traversal prevention.
Database	MySQL 8.0, SQLAlchemy, PyMySQL	Relational persistence for users, files, folders, shares, AI metadata, activity logs.
ML Engine	Scikit-learn, Random Forest, Joblib	Feature encoding, real-time file classification & suspicious upload scoring.
GenAI Module	Google Gemini 3.6 Flash, PyPDF, Docx	Document parsing, multimodal LLM execution, structured JSON summaries & tags.

4. Tools & Technologies Used

- **Programming Languages:** Python 3.11, JavaScript (ES6+), HTML5, CSS3, SQL
- **Backend Framework:** FastAPI, Starlette, Uvicorn ASGI Server

- **Database & ORM:** MySQL 8.0, SQLAlchemy 2.0, PyMySQL
- **Machine Learning:** Scikit-learn (RandomForestClassifier), Joblib, Pandas, NumPy, Matplotlib, Seaborn
- **Generative AI:** Google GenAI SDK (google-genai), Gemini 3.6 Flash LLM
- **Security & Auth:** Passlib (bcrypt), PyJWT / python-jose, Secrets (UUID4)
- **Development Tools:** VS Code, Git, Jupyter Notebook, ReportLab, Python-Docx

5. Machine Learning Implementation Details

The project implements two distinct Random Forest Classifier models saved as serialized joblib binaries:

1. **File Category Classifier:** Trained on metadata extracted from thousands of files to categorize uploads into Document, Image, Video, Audio, or Code.
2. **Suspicious Upload Anomaly Detector:** Trained on user behavioral patterns (upload frequency, night-time upload flags, file size anomalies, unknown extensions, suspicious keyword indicators). Evaluates uploads in real time to assign safety confidence scores.

6. Generative AI Implementation Details

The GenAI module extracts content from uploaded documents (using PyPDF, python-docx, or raw text streams) or passes image/audio/video files directly to Google Gemini 3.6 Flash. Prompts are structured to return JSON with Smart Overview, Detailed File Description, Smart Categorical Tags, and Key Content Insights.